

Crypto Investment Analyzer - REST API Client Application

1. Application Description: The Crypto Investment Analyzer is a Java command-line application that consumes the CoinGecko REST API to fetch real-time cryptocurrency market data and provides automated investment recommendations based on a weighted scoring algorithm. The application analyzes Bitcoin, Ethereum, BNB, Cardano, and Solana by default, with support for custom coin selection and trending coin discovery.

2. Technical Architecture

- **Language:** Java 11 with built-in `java.net.http.HttpClient` (no external dependencies)
- **JSON Parsing:** Manual regex-based parser developed specifically for CoinGecko API responses
- **API Provider:** CoinGecko Free API which requires no authentication or API key
- **Program Modules:** Config, MarketData, InvestmentAdvice, ApiClient, JsonParser, ScoringEngine, CryptoAnalyzer

3. REST Principles Demonstrated

- **Statelessness:** Each API request contains all required parameters including coin ID and currency
- **Resource-Oriented:** Each cryptocurrency is modeled as a unique resource at the `/coins/{id}` endpoint
- **Uniform Interface:** Standard HTTP GET method with proper status codes (200 OK, 404 Not Found, 429 Too Many Requests)
- **Cacheability:** Rate limiting with 25 calls per minute and 3-second delays between requests
- **Richardson Maturity Model:** CoinGecko API operates at Level 2 (Resources plus HTTP Verbs)

4. Scoring Algorithm (0-100 points)

Factor	Points	Description
Momentum	25	Positive 24-hour price change indicates upward trend
Market Cap	20	Larger capitalization suggests lower risk and stability
ATH Distance	20	Greater distance from all-time high represents value opportunity
Liquidity	20	Volume to market cap ratio indicates ease of entry and exit
Rank	15	Higher rank (lower number) indicates more established project

Recommendation Scale: 80+ STRONG BUY | 60-79 BUY | 40-59 HOLD | 20-39 WATCH | 0-19 SELL

5. How to Run

- Compile all source files: `javac -d . src/com/crypto/analyzer/*.java`
- Execute the main program: `java -cp . com.crypto.analyzer.CryptoAnalyzer`
- Type 'y' for default coins or 'n' to enter custom IDs. Output saves to `crypto_analysis_output.txt`.

6. Conclusion: This application demonstrates core Service-Oriented Architecture principles including autonomous services, loose coupling, and message-based communication via HTTP requests. The implementation follows the Google Java Style Guide with 80-character line limits, comprehensive Javadoc comments, and clean modular design. See `crypto_analysis_output.txt` for the complete sample output from a successful program execution.